

O Processador: Caminho de Dados e Controle

"Em um assunto importante, nenhum detalhe é pequeno", Provérbio francês

Conteúdo

1	Introdução	2
2	Convenções lógicas de projeto	3
2.1	Metodologia de clocking	4
3	Construindo um caminho de dados	5
4	Um esquema de implementação simples	9
4.1	O controle da ALU	9
4.2	Projetando a unidade de controle principal	10
4.3	Operações do caminho de dados	13
4.4	Finalizando o controle	15
5	Um esquema de implementação Multiciclo	17
5.1	Dividindo a execução da instrução em ciclos de clock	21
5.2	Definindo o controle	23

1 Introdução

⇒ Analisaremos uma implementação que inclui um subconjunto do conjunto de instruções MIPS básico:

1. Instruções de referência à memória: lw e sw
2. Instruções lógicas e aritméticas: add, sub, and, or e slt
3. Instruções de desvio: beq e jump(j)

⇒ Para cada instrução, as duas primeiras etapas são idênticas:

1. Enviar o contador de programa (PC) à memória que contém o código (Memória de Instruções) e buscar a instrução dessa memória
2. Ler um ou mais registradores, usando campos da instrução para selecionar os registradores a serem lidos → para a instrução lw, precisamos ler apenas um registrador, mas a maioria das outras instruções exige a leitura de dois registradores

⇒ Todas as classes de instrução, exceto jump, usam a unidade lógica e aritmética (ALU) após a leitura dos registradores

⇓

As instruções de referência à memória usam a ALU para o cálculo de endereço, as instruções lógicas e aritméticas para a execução da operação e as instruções de desvio para comparação

⇒ Após usar a ALU, as ações necessárias para completar várias classes de instrução diferem

⇒ Uma instrução de referência à memória precisará acessá-la a fim de ler dados para um load ou escrever dados para um store

⇒ Uma instrução lógica e aritmética precisa escrever os dados obtidos da ALU de volta a um registrador

⇒ Para uma instrução de desvio, podemos ter de mudar o próximo endereço de instrução com base na comparação → caso contrário, o PC deve ser incrementado em 4 a fim de chegar ao endereço da próxima instrução

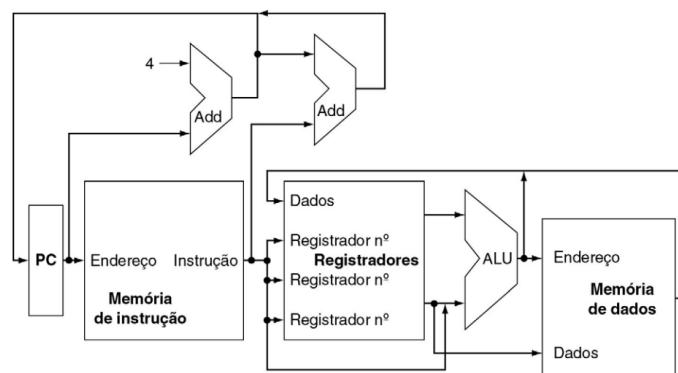


Figura 1: Uma visão abstrata da implementação do subconjunto MIPS mostrando as principais unidades funcionais e as principais conexões entre elas

⇒ O valor escrito no PC pode vir de dois somadores; os dados para serem escritos no Banco de Registradores podem vir da ALU ou da memória de dados; e a segunda entrada da ALU pode vir de um registrador ou do campo imediato da instrução

- ⇒ Precisamos adicionar um elemento que escolha dentre as diversas origens e conduza uma dessas origens a seu destino → a seleção é feita com um multiplexador (mux) que é um seletor de dados
- ⇒ A memória de dados precisa ser lida em um load e ser escrita em um store
- ⇒ O Banco de Registradores precisa ser escrito em uma instrução load e em uma instrução lógica ou aritmética
- ⇒ A ALU precisa realizar uma de várias operações
- ⇒ A unidade de controle, que tem a instrução como uma entrada, é usada para determinar como definir as linhas de controle para as unidades funcionais e dois dos multiplexadores
- ⇒ O terceiro multiplexador – que determina se $PC + 4$ ou o endereço de destino do desvio é escrito no PC – é definido com base na saída zero da ALU, usada para realizar a comparação da instrução beq

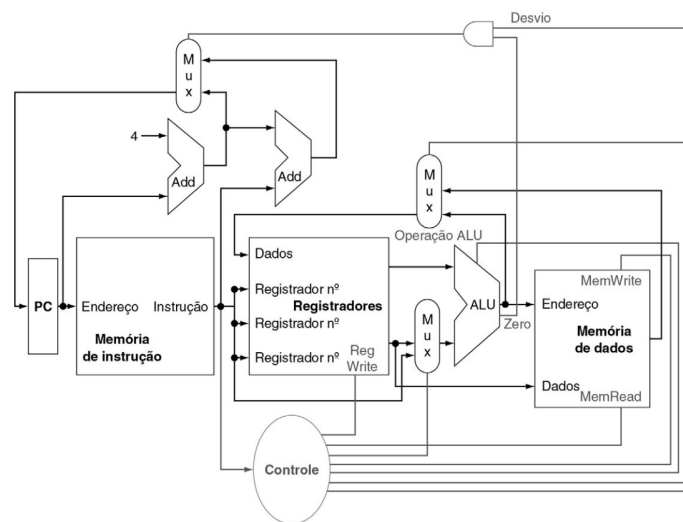


Figura 2: A implementação básica do subconjunto MIPS incluindo as linhas de controle e os multiplexadores necessários

2 Convenções lógicas de projeto

- ⇒ Os elementos do caminho de dados na implementação MIPS consistem em dois tipos diferentes de elementos lógicos: aqueles que operam nos valores dos dados (combinacionais) e os que contêm estado (sequenciais)
- ⇒ **Combinacionais**: Dado um conjunto de entradas, ele sempre produz a mesma saída pois não possui qualquer armazenamento interno
- ⇒ **Sequenciais**: Apresentam algum armazenamento interno e também são chamados de **Elementos de estado**
- ⇒ Um elemento de estado possui pelo menos duas entradas e uma saída:

As entradas necessárias são os valores dos dados a serem escritos no elemento e o clock que determina quando o valor dos dados deve ser escrito

A(s) saída(s) que pode(m) fornecer o valor escrito em um ciclo de clock anterior (ou um outro dado selecionado)

⇒ Além dos flip-flops, nossa implementação MIPS também usa dois outros tipos de elementos de estado: memórias e registradores → como vemos na figura 2

⇒ O clock é usado para determinar quando se deve escrever no elemento de estado; um elemento de estado pode ser lido a qualquer momento

⇒ Os componentes lógicos que contêm estado também são chamados de sequenciais porque suas saídas dependem de suas entradas e do conteúdo do estado interno

⇒ A saída da unidade funcional representando os registradores (Banco de Registradores) depende dos números de registrador fornecidos e do que foi escrito nos registradores anteriormente

2.1 Metodologia de clocking

⇒ Uma metodologia de clocking define quando os sinais podem ser lidos e quando podem ser escritos

⇒ Ela é importante para especificar a sincronização das leituras e escritas porque, se um sinal fosse escrito ao mesmo tempo em que fosse lido, o valor da leitura poderia corresponder ao valor antigo, ao valor recém-escrito ou mesmo alguma combinação dos dois! → não podemos tolerar essa imprevisibilidade!

⇒ Para simplificar, consideraremos uma metodologia de sincronização acionada por transição



Quaisquer valores armazenados em um elemento lógico sequencial são atualizados apenas em uma transição do clock

⇒ Como apenas os elementos de estado podem armazenar valores de dados, qualquer coleção de lógica combinatória precisa ter suas entradas vindo de um conjunto de elementos de estado e suas saídas escritas em um conjunto de elementos de estado → escritas são feitas caso os dados das saídas sejam necessários no(s) próximo(s) ciclo de clock

⇒ As entradas são valores escritos em um ciclo de clock anterior, enquanto as saídas são valores que podem ser usados em um ciclo de clock seguinte

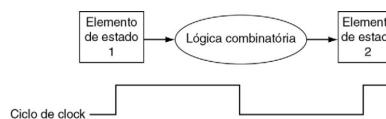


Figura 3: A lógica combinatória, os elementos de estado e o clock estão intimamente relacionados

⇒ O tempo necessário para os sinais alcançarem o elemento 2 (como na figura acima) define a duração do ciclo de clock

⇒ Tanto o **signal de clock** quanto o **signal de controle de escrita** são entradas → o elemento de estado só é alterado quando o sinal de controle de escrita está ativo e ocorre uma transição do clock

⇒ Uma metodologia acionada por transição permite ler o conteúdo de um registrador, enviar o valor por meio de alguma lógica combinatória e escrever nesse registrador no mesmo ciclo de clock



Figura 4: Uma metodologia acionada por transição permite que um elemento de estado seja lido e escrito no mesmo ciclo de clock sem criar uma disputa que poderia levar a valores de dados indeterminados

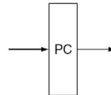
⇒ Para a arquitetura MIPS de 32 bits, quase todos esses elementos de estado e lógicos terão entradas e saídas contendo 32 bits de extensão → extensão da maioria dos dados manipulados pelo processador

3 Construindo um caminho de dados

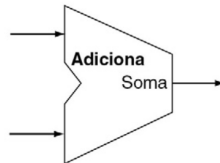
⇒ **Unidade de memória de Instruções:** armazena as instruções de um programa e fornece instruções dado um endereço



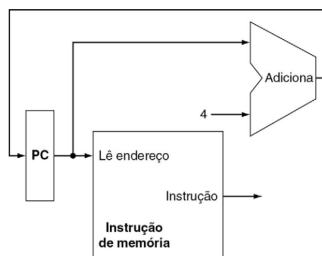
⇒ **Contador de programa (PC):** registrador que contém o endereço da instrução atual



⇒ Precisaremos de um **somador** a fim de incrementar o PC para o endereço da próxima instrução: pode ser construído a partir da ALU simplesmente interligando as linhas de controle de modo que o controle sempre especifique uma operação de adição



⇒ Combinando os três elementos da figura acima para formar um caminho de dados que busca instruções e incrementa o PC de modo a obter o endereço da próxima instrução sequencial:



⇒ Instruções do tipo R: Todas elas leem dois registradores, realizam uma operação na ALU com o conteúdo dos registradores e escrevem o resultado em um registrador

Nome	Campos						Comentários
Tamanho do campo	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	Todas as instruções do MIPS têm 32 bits
Formato R	op	rs	rt	rd	shamt	funct	Formato das instruções aritméticas
Formato I	op	rs	rt	endereço/imediato			Formato imediato para transferências e desvios
Formato J	op	endereço de destino					Formato das instruções de jump

⇒ **Banco de Registradores:** coleção de registradores em que qualquer registrador pode ser lido ou escrito especificando o número do registrador → armazena o estado (dados) de todos os registradores

⇒ Devido ao fato das instruções de formato R terem 3 operandos de registrador, precisaremos ler 2 palavras de dados do Banco de Registradores e escrever 1 palavra de dados no Banco de Registradores para cada instrução

⇒ A fim de que cada palavra de dados seja lida dos registradores, precisamos de uma entrada no Banco de Registradores que especifique o número do registrador a ser lido e uma saída do Banco de Registradores que

conduzirá o valor lido dos registradores

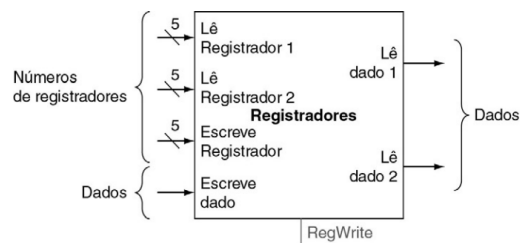
⇒ Para escrever uma palavra de dados, precisaremos de duas entradas: 1 para especificar o número do registrador a ser escrito e 1 para fornecer os dados a serem escritos no registrador

⇒ O Banco de Registradores sempre gera como saída o conteúdo de quaisquer números de registrador que estejam nas entradas "Lê Registrador"

⇒ As escritas, entretanto, são controladas pelo sinal de controle de escrita (**RegWrite**), que precisa ser ativo para que uma escrita ocorra na transição do clock

⇒ Precisamos de um total de quatro entradas (3 para números de registrador e uma para dados) e 2 saídas (ambas para dados)

⇒ As entradas de número de registrador possuem 5 bits de largura para especificar um dos 32 registradores ($32 = 2^5$), enquanto a entrada de dados e os 2 barramentos de saída de dados possuem 32 bits de largura cada um



⇒ **ALU**: usa duas entradas de 32 bits e produz um resultado de 32 bits, bem como um sinal de 1 bit se o resultado for 0 (para beq) - Saída Zero

⇒ O sinal de controle de 4 bits da ALU é descrito em detalhes mais a frente



⇒ A seguir, considere as instruções MIPS load word e store word, que possuem o formato `lw $t1, offsetValue($t2)` ou `sw $t1, offsetValue($t2)`

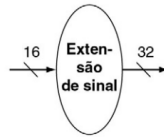
⇒ Essas instruções calculam um endereço de memória somando o registrador de base, que é \$t2, com o campo offset de 16 bits com sinal contido na instrução

⇒ Se a instrução for um store, o dado a ser armazenado também precisará ser lido do Banco de Registradores em que reside, em \$t1

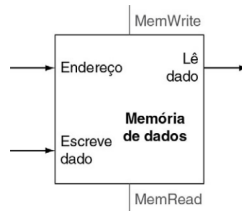
⇒ Se a instrução for um load, o dado lido da memória precisará ser escrito no Banco de Registradores no registrador especificado, que é \$t1.

⇒ Consequentemente, precisaremos do Banco de Registradores e da ALU vistos anteriormente

⇒ Precisaremos de uma **unidade a fim de estender o sinal do campo offset de 16 bits** da instrução para um valor com sinal de 32 bits, e de uma unidade de memória da qual ler ou na qual escrever



⇒ A **Memória de dados** precisa ser escrita em instruções store e lida em uma instrução load → ela tem sinais de controle de leitura (**MemRead**) ativados em loads e de escrita (**MemWrite**) ativados em stores, uma entrada de endereço e uma entrada para os dados serem escritos na memória



⇒ A instrução beq possui 3 operandos: 2 registradores comparados para igualdade e um offset de 16 bits para calcular o endereço de destino do desvio

⇒ Sua forma é beq \$t1, \$t2, offset → para implementar essa instrução, precisamos calcular o endereço de destino do desvio somando o campo offset estendido com sinal da instrução + o PC

⇒ A base para o cálculo do endereço de desvio é o endereço da instrução seguinte ao desvio ($PC + 4 + \text{offset}$)

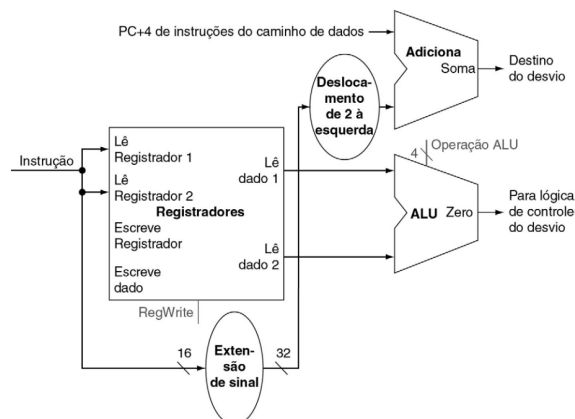
⇒ Como calculamos $PC + 4$ (o endereço da próxima instrução) no caminho de dados para busca de instruções, é fácil usar esse valor como a base para calcular o endereço de destino do desvio

⇒ O campo offset é **deslocado 2 bits** para a esquerda de modo que seja um offset de uma palavra (32 bits) → esse deslocamento aumenta a faixa efetiva do campo offset por um fator de 4 vezes

⇒ Além de calcular o endereço de destino do desvio, também precisamos determinar se a próxima instrução é a instrução que acompanha sequencialmente ou a instrução no endereço de destino do desvio → precisamos determinar se o desvio é tomado ou não

⇒ Quando a condição é verdadeira (isto é, os operandos são iguais), o endereço de destino do desvio se torna o novo PC e dizemos que o desvio é tomado

⇒ Se os operandos não forem iguais, o PC incrementado ($PC + 4$) deve substituir o PC atual (exatamente como para qualquer outra instrução normal) → dizemos que o desvio é não tomado



⇒ Como essa ALU fornece um sinal de saída que indica se o resultado era 0, podemos enviar os dois operandos de registrador para a ALU com o conjunto de controle de modo a fazer uma subtração (se a subtração resultar em 0, os dois registradores são iguais → SaídaZeroALU = 1)

⇒ Se o sinal Zero da ALU estiver ativo, sabemos que os dois valores são iguais

⇒ Embora a saída de Zero sempre sinalize quando o resultado é 0, nós a estaremos usando apenas para implementar o teste de igualdade dos desvios

⇒ Agora que examinamos os componentes do caminho de dados necessários para as classes de instrução individualmente, podemos combiná-los em um único caminho de dados e acrescentar o controle para completar a implementação

⇒ Nenhum recurso do caminho de dados pode ser usado mais de uma vez por instrução e, portanto, qualquer elemento necessário mais de uma vez precisa ser duplicado

⇒ Para compartilhar um elemento do caminho de dados entre duas classes de instrução diferentes, talvez tenhamos de permitir múltiplas conexões com a entrada de um elemento → usamos um mux e um sinal de controle para selecionar entre as múltiplas entradas

⇒ Exemplo:

As operações do caminho de dados das instruções lógicas e aritméticas (tipo R) e das instruções de acesso à memória são muito semelhantes → As principais diferenças são as seguintes:

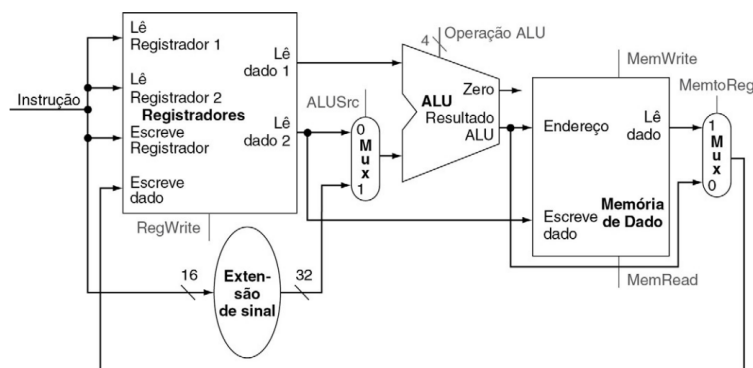
As instruções lógicas e aritméticas usam a ALU com as entradas vindas de dois registradores; As instruções de acesso à memória também podem usar a ALU para fazer o cálculo do endereço, embora a segunda entrada seja o campo offset de 16 bits com sinal estendido da instrução

O valor armazenado em um registrador de destino vem da ALU (para uma instrução tipo R) ou da memória (para um load)

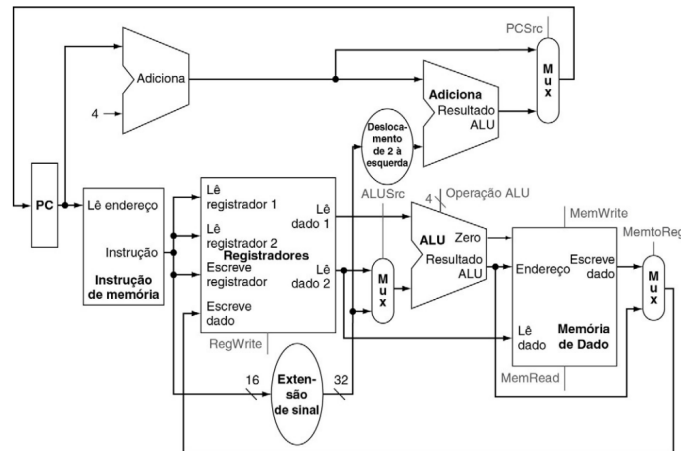
Mostre como construir um caminho de dados para a parte operacional das instruções de acesso à memória e das instruções lógicas e aritméticas, que use um único Banco de Registradores e uma única ALU para manipular os dois tipos de instrução, incluindo quaisquer multiplexadores necessários

Para criar um caminho de dados com apenas um único Banco de Registradores e uma única ALU, precisamos suportar duas origens diferentes para a segunda entrada da ALU, bem como duas origens diferentes para os dados armazenados no Banco de Registradores

Portanto, um multiplexador é colocado na entrada da ALU (controlado por **ALUSrc**) e outro na entrada de dados para o Banco de Registradores (controlado por **MemtoReg**). A figura abaixo mostra a parte operacional do caminho de dados combinado:



4 Um esquema de implementação simples



4.1 O controle da ALU

Linhas de controle da ALU	Função
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

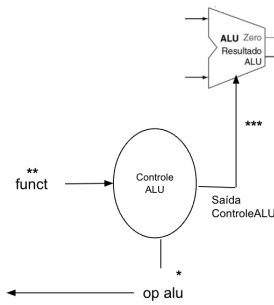
- ⇒ Dependendo da classe de instrução, a ALU precisará realizar uma dessas cinco primeiras funções
- ⇒ Para as instruções load word e store word, usamos a ALU para calcular o endereço de memória por adição
- ⇒ Para instruções tipo R, a ALU precisa realizar uma das cinco ações (and, or, subtract, add ou set on less than), dependendo do valor do campo **funct** (function – "função") de 6 bits nos bits menos significativos da instrução
- ⇒ Para branch equal (beq), a ALU precisa realizar uma subtração → se o resultado for 0, os valores são iguais
- ⇒ Podemos gerar a entrada do controle da ALU de 4 bits usando uma pequena unidade de controle que tenha como entradas o campo funct da instrução e um campo control de 2 bits, que chamamos de OpALU
- ⇒ **OpALU** indica se a operação a ser realizada deve ser add (00) para loads e stores, subtract (01) para beq ou determinada pela operação codificada no campo funct (10)
- ⇒ A saída da unidade de controle da ALU é um sinal de 4 bits que controla diretamente a ALU gerando uma das combinações de 4 bits mostradas anteriormente

Opcode da instrução	OpALU	Operação da instrução	Campo funct	Ação da ALU desejada	Entrada do controle da ALU
LW	00	load word		add	0010
SW	00	store word	XXXXXX	add	0010
Branch equal	01	branch equal	XXXXXX	subtract	0110
tipo R	10	add	100000	add	0010
tipo R	10	subtract	100010	subtract	0110
tipo R	10	AND	100100	AND	0000
tipo R	10	OR	100101	OR	0001
tipo R	10	set on less than	101010	set on less than	0111

Opcodo da instrução	*		**		***	
	OpALU	Operação da instrução	Campo funct	Ação da ALU desejada	Entrada de controle da ALU	
LW	00	load word		add	0010	
SW	00	store word	XXXXXX	add	0010	
Branch equal	01	branch equal	XXXXXX	subtract	0110	
tipo R	10	add	100000	add	0010	
tipo R	10	subtract	100010	subtract	0110	
tipo R	10	AND	100100	AND	0000	
tipo R	10	OR	100101	OR	0001	
tipo R	10	set on less than	101010	set on less than	0111	

Linhas de controle da ALU	Função
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

00 (add): lw/sw
01 (sub): beq
10 (funct): tipo R



⇒ Esse estilo de usar vários níveis de decodificação – ou seja, a unidade de controle principal gera os bits de OpALU, que, então, são usados como entrada para o controle da ALU que gera os sinais reais para controlar a ALU – é uma técnica de implementação comum

⇒ Usar várias unidades de controle menores também pode aumentar a velocidade da unidade de controle

⇒ Há várias maneiras diferentes de implementar o mapeamento do campo OpALU de 2 bits e do campo funct de 6 bits para os 4 bits de controle de operação da ALU

⇒ Como uma etapa no projeto dessa lógica, é útil criar uma tabela verdade para as combinações interessantes do campo de código funct e dos bits de OpALU, como fizemos na figura abaixo; essa tabela verdade mostra como o controle da ALU de 4 bits é definido de acordo com esses dois campos de entrada:

OpALU		Campo funct						Operação
OpALU1	OpALU 2	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

4.2 Projetando a unidade de controle principal

Campo	0	rs	rt	rd	shamt	funct
Posição de bit	31:26	25:21	20:16	15:11	10:6	5:0
a. Instrução do tipo R						
Campo	35 ou 43	rs	rt	address		
Posição de bit	31:26	25:21	20:16	15:0		
b. Carrega ou armazena instrução						
Campo	4	rs	rt	address		
Posição de bit	31:26	25:21	20:16	15:0		
c. Instrução de desvio						

■ Campo op (opcode) está sempre contido nos bits 31:26 → nos referimos a ele como Op[5:0]

■ Os dois registradores a serem lidos sempre são especificados pelos campos rs e rt, nas posições 25:21 e 20:16 → isso é verdade para as instruções tipo R, branch equal e store

■ Registrador base para as instruções load e store está sempre nas posições de bit 25:21 (rs)

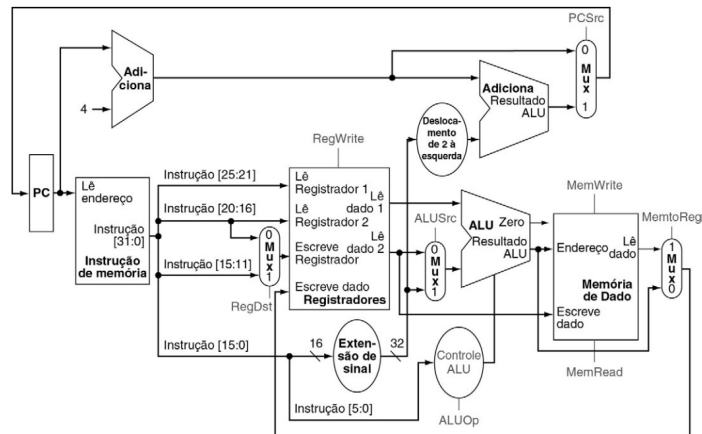
■ Offset de 16 bits para branch equal, load e store está sempre nas posições 15:0

■ Registrador de destino está em um de dois lugares: para um load, ele está nas posições 20:16 (rt), enquanto para uma instrução tipo R, ele está nas posições 15:11 (rd)

⇒ Portanto, precisaremos incluir um mux a fim de selecionar que campo da instrução será usado para indicar o número de registrador a ser escrito (controlado pelo sinal de controle **RegDst**)

⇒ A figura abaixo mostra essas adições, além do bloco de controle da ALU, os sinais de escrita para elementos de estado, o sinal de leitura para a memória de dados e os sinais de controle para os multiplexadores

⇒ Como todos os multiplexadores possuem duas entradas, cada um deles requer uma única linha de controle (1 bit) → escolhe 0 ou 1



⇒ A unidade de controle pode definir todos menos um dos sinais de controle unicamente com base no campo opcode da instrução → exceção é a linha de controle **PCSrc** (Cuidado, no livro esse nome foi traduzido para OrigPC!)

⇒ Essa linha de controle deve ser ativada se a instrução for branch on equal e a saída Zero da ALU, usada para comparação de igualdade, for verdadeira

⇒ Para gerar o sinal **PCSrc**, precisaremos realizar um AND de um sinal da unidade de controle, que chamamos **Branch**, com o sinal Zero da ALU

⇒ Esses nove sinais de controle (7 da figura acima e 2 para OpALU) podem agora ser definidos baseados nos 6 sinais de entrada da unidade de controle, que são os bits de opcode 31 a 26

Nome do sinal	Efeito quando inativo	Efeito quando ativo
RegDst	O número do registrador destino para entrada Registrador para escrita vem do campo rt (bits 20:16).	O número do registrador destino para a entrada Registrador para escrita vem do campo rd (bits 15:11).
RegWrite	Nenhum.	O registrador na entrada Registrador para escrita é escrito com o valor da entrada Dados para escrita.
ALUSrc	O segundo operando da ALU vem da segunda saída do banco de registradores (Dados da leitura 2).	O segundo operando da ALU consiste nos 16 bits mais baixos da instrução com sinal estendido.
PCSrc	O PC é substituído pela saída do somador que calcula o valor de PC + 4.	O PC é substituído pela saída do somador que calcula o destino do desvio.
MemRead	Nenhum.	O conteúdo da memória de dados designado pela entrada Endereço é colocado na saída Dados da leitura.
MemWrite	Nenhum.	O conteúdo da memória de dados designado pela entrada Endereço é substituído pelo valor na entrada Dados para escrita.
MemtoReg	O valor enviado à entrada Dados para escrita do banco de registradores vem da ALU.	O valor enviado à entrada Dados para escrita do banco de registradores vem da memória de dados.

*Os nomes dos sinais das tabelas no resumo foram alterados do livro da versão do livro da 4ed em português tendo em vista que os nomes dos sinais nos esquemáticos do caminho de dados estão "em inglês" e as tabelas explicando os sinais no livro foram traduzidas para português; no resumo na parte do uniciclo estão os nomes dos sinais em inglês para ficar mais coerente!

Instrução	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
formato R	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

Figura 5: A definição das linhas de controle é completamente determinada pelos campos opcode da instrução

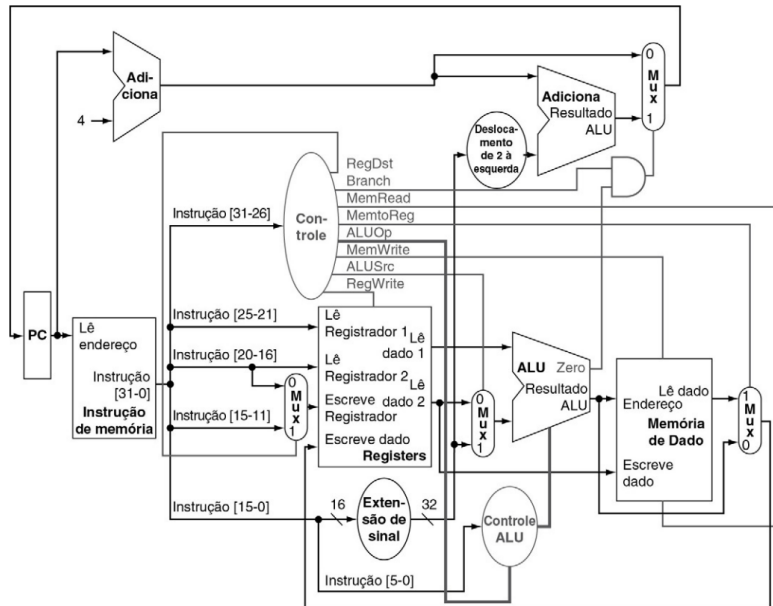


Figura 6: Caminho de dados com sinais especificados

⇒ É importante ressaltar que para a execução correta das instruções os sinais de controle tem de estar precisamente como na tabela da figura 5

⇒ Nem sempre um sinal pode ser Don't Care → ex: para instruções que não são de acesso à memória temos de desativar MemRead e MemWrite, pois se fosse Don't Care estaríamos sujeitos a escrever dados errados na memória ou ler dados que não são necessários

⇒ Outros podem ser Don't Care → ex: RegDst em sw/beq, pois qualquer valor escolhido para o mux que esse sinal controla não vai alterar nenhum elemento de estado de forma incorreta, e não precisamos de dados que esse mux seleciona para essas 2 instruções (sw/beq) já que nelas não fazemos uma escrita no Banco de Registradores (a escrita só é feita quando RegWrite = 1, que não é o caso de sw/beq)

***Os nomes dos sinais das tabelas no resumo foram alterados do livro da versão do livro da 4ed em português tendo em vista que os nomes dos sinais nos esquemáticos do caminho de dados estão "em inglês" e as tabelas explicando os sinais no livro foram traduzidas para português; no resumo na parte do uniciclo estão os nomes dos sinais em inglês para ficar mais coerente!**

4.3 Operações do caminho de dados

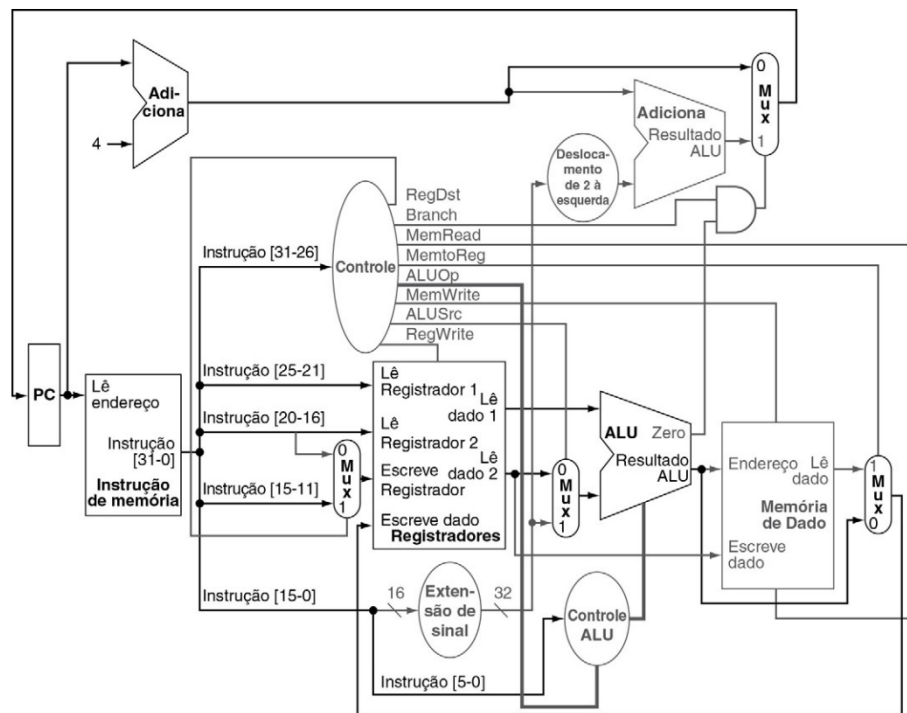


Figura 7: Exemplo Instrução Tipo R

add \$t1, \$t2, \$t3

1. A instrução é buscada e o PC é incrementado
2. Dois registradores, \$t2 e \$t3, são lidos do Banco de Registradores, e a unidade de controle principal calcula a definição das linhas de controle também durante essa etapa
3. A ALU opera nos dados lidos do Banco de Registradores, usando o código de função (bits 5:0, que é o campo funct, da instrução) para gerar a função da ALU
4. O resultado da ALU é escrito no Banco de Registradores usando os bits 15:11 da instrução para selecionar o registrador de destino (\$t1)

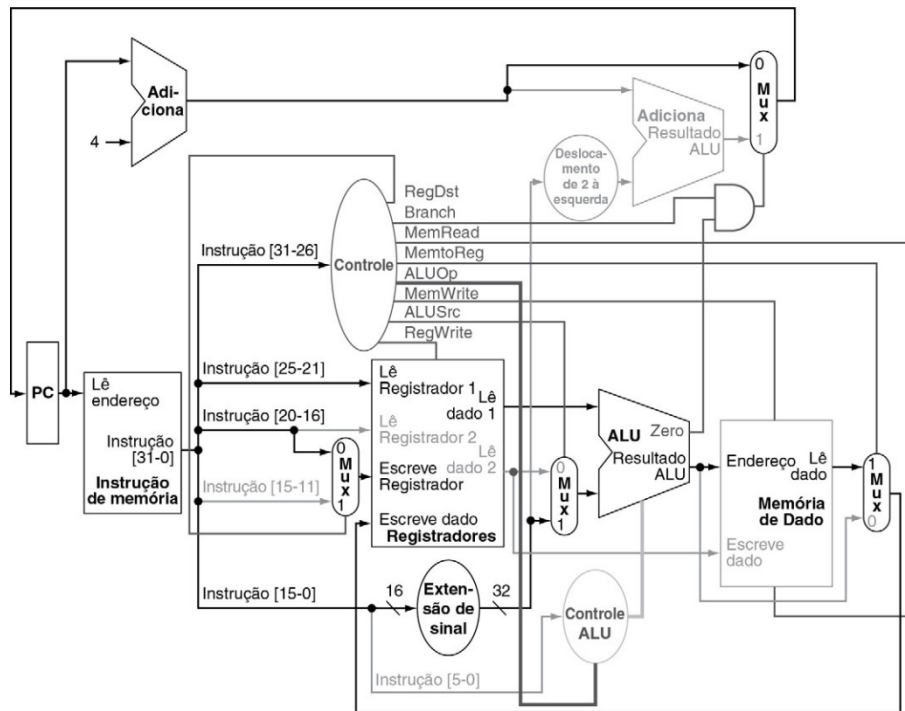


Figura 8: Exemplo Instrução Load

lw \$t1, offset(\$t2)

1. Uma instrução é buscada da memória de instruções e o PC é incrementado
2. Um valor de registrador (\$t2) é lido do Banco de Registradores
3. A ALU calcula a soma do valor lido do Banco de Registradores com os 16 bits menos significativos com sinal estendido da instrução (offset)
4. A soma da ALU é usada como o endereço para a memória de dados
5. Os dados da unidade de memória são escritos no Banco de Registradores; o registrador de destino é fornecido pelos bits 20:16 da instrução (\$t1)

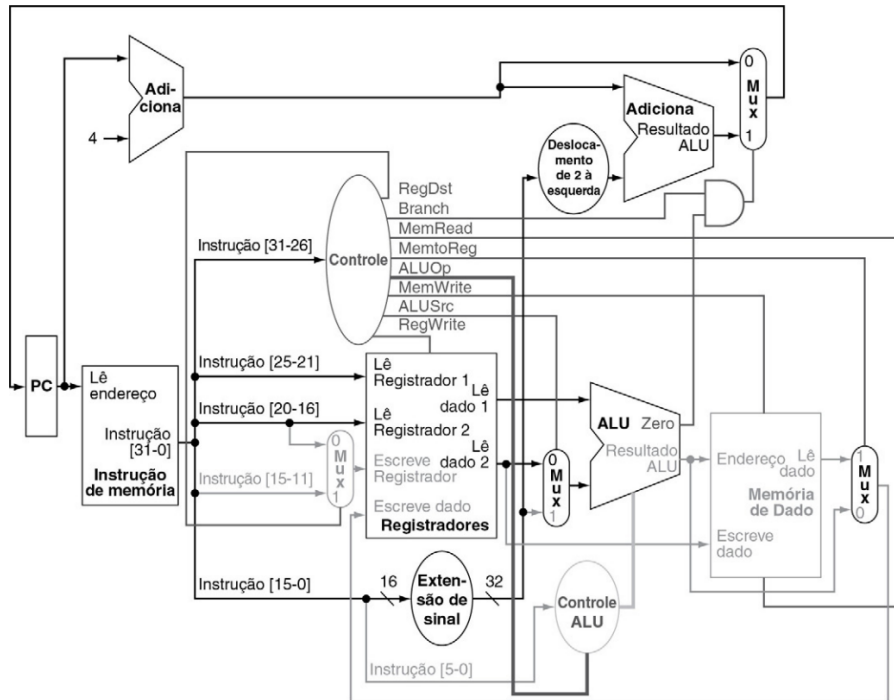


Figura 9: Exemplo Instrução beq

beq \$t1,\$t2,offset

1. Uma instrução é buscada da memória de instruções e o PC é incrementado
2. Dois registradores, \$t1 e \$t2, são lidos do Banco de Registradores
3. A ALU realiza uma subtração dos valores de dados lidos do Banco de Registradores. O valor de PC + 4 é somado aos 16 bits menos significativos com sinal estendido (offset) deslocados de dois para a esquerda; o resultado é o endereço de destino do desvio
4. O resultado Zero da ALU é usado para decidir qual resultado do somador deve ser armazenado no PC

4.4 Finalizando o controle

⇒ A Figura abaixo mostra a lógica na unidade de controle como uma grande tabela verdade que combina todas as saídas e que usa os bits de opcode como entradas → especifica completamente a função de controle, e podemos implementá-la diretamente em portas lógicas de uma maneira automatizada

Entrada ou saída	Nome do sinal	formato R	lw	sw	beq
Entradas	Op5	0	1	1	0
	Op4	0	0	0	0
	Op3	0	0	1	0
	Op2	0	0	0	1
	Op1	0	1	1	0
	Op0	0	1	1	0
	Op0	0	1	1	0
Saídas	RegDst	1	0	X	X
	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

⇒ Exemplo:

Estenda o caminho de dados e o controle da figura 6 para incluir a instrução jump. Descreva como definir quaisquer novas linhas de controle

A instrução jump, mostrada na figura abaixo, se parece um pouco com uma instrução branch, mas calcula o PC de destino de maneira diferente e não é condicional

Como um branch, os 2 bits menos significativos de um endereço jump são sempre 00_{bin}

Os próximos 26 bits menos significativos desse endereço de 32 bits vêm do campo imediato de 26 bits na instrução

Os 4 bits superiores do endereço que deve substituir o PC vêm do PC da instrução jump mais 4. Portanto, podemos implementar um jump armazenando no PC a concatenação de:

- Os 4 bits superiores do PC atual + 4 (esses são bits 31:28 do endereço da instrução imediatamente seguinte)
- Campo de 26 bits imediato da instrução jump
- Os bits 00_{bin}

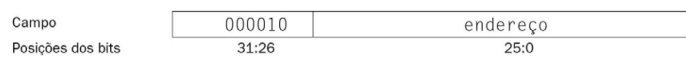


Figura 10: Formato de instrução para a instrução jump (opcode = 2). O endereço de destino para uma instrução jump é formado pela concatenação dos 4 bits superiores do PC atual + 4 com o campo endereço de 26 bits na instrução jump e pela adição de 00 como os dois bits menos significativos.

A figura abaixo mostra a adição do componente para jump à figura do Caminho de dados com Sinais:

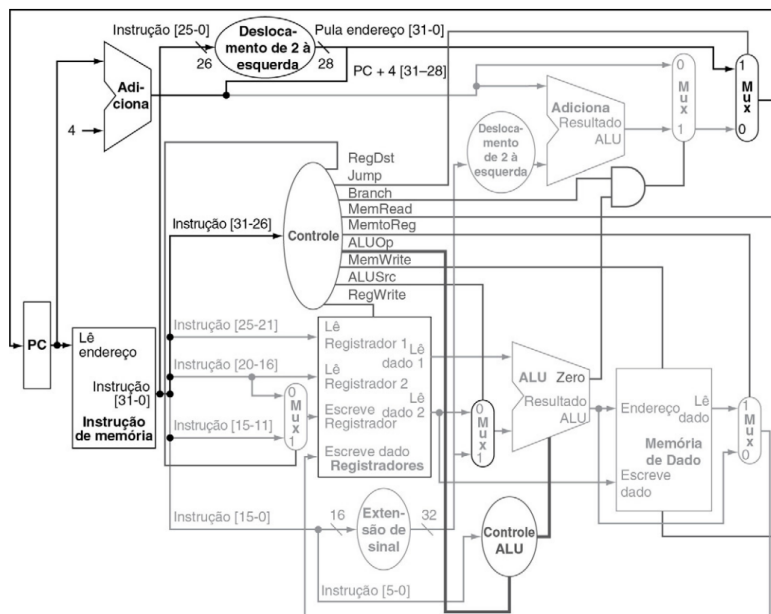


Figura 11: Caminho de dados com suporte a jump

Um outro multiplexador é usado na seleção da origem para o novo valor do PC, que pode ser o PC incrementado ($PC + 4$), o PC de destino de um branch ou o PC de destino de um jump

Um sinal de controle adicional é necessário para o multiplexador adicional

Esse sinal de controle, chamado **Jump**, é ativado apenas quando a instrução é um jump – ou seja, quando o opcode é 2

5 Um esquema de implementação Multiciclo

***Os nomes dos sinais nesse capítulo estão todos na versão traduzida nas tabelas e esquemáticos do caminho de dados, tomar cuidado, pois no uniclo todos os sinais estão "em ingles" ex: LeMem = MemRead; EscreveMem = MemWrite;**

⇒ Em um exemplo anterior, dividimos cada instrução em uma série de etapas correspondentes às operações das unidades funcionais necessárias → podemos usar essas etapas para criar uma implementação multiciclo

⇒ Em uma implementação multiciclo, cada etapa levará 1 ciclo de clock

⇒ Esta implementação permite que uma unidade funcional seja usada mais de 1 vez por instrução, desde que seja usada em ciclos de clock diferentes → permite uma maior reusabilidade do hardware

⇒ Abaixo temos uma versão abstrata do caminho de dados do multiciclo:

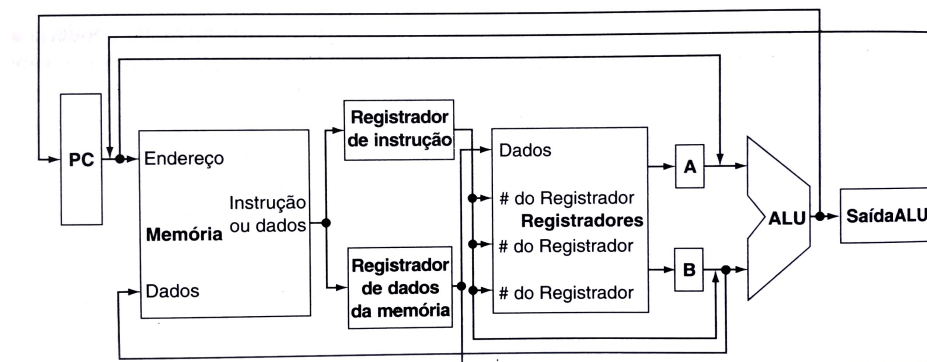


Figura 12: A visão de alto nível do caminho de dados multiciclo

⇒ Comparando com o caminho de dados da versão uniclo, vemos as seguintes diferenças:

- Uma única unidade de memória é usada para instruções e para dados
- Existe uma única ALU em vez de 1 ALU e 2 somadores
- Um ou mais registradores são adicionados após cada unidade funcional para conter a saída dessa unidade até o valor ser usado em um ciclo de clock subsequente

⇒ No final de um ciclo de clock, todos os dados usados nos ciclos de clock subsequentes precisam ser armazenados em um elemento de estado

⇒ Os dados usados pelas instruções subsequentes em um ciclo de clock posterior são armazenados em um dos elementos de estados visíveis ao programador, como já fazíamos: o Banco de Registradores, o PC ou a Memória

⇒ Por outro lado, os dados usados pela mesma instrução precisam ser armazenados em um desses registradores adicionais

⇒ Consideramos que podemos acomodar em um ciclo de clock: 1 acesso à memória, 1 acesso ao Banco de Registradores (2 leituras e 1 escrita) ou 1 operação da ALU → quaisquer dados produzidos por uma dessas 3 unidades funcionais precisam ser salvos em um registrador temporário para uso no ciclo posterior → se eles não fossem salvos, poderia haver a possibilidade de uma disputa de sincronização, levando ao uso de um valor incorreto

■ **Registrador de instrução (IR)** e o **Registrador de dados da Memória(MDR)**: salvam a saída para a leitura de uma instrução e uma leitura de dados; usamos dois registradores pois, como veremos, ambos os valores são necessários durante o mesmo ciclo de clock

- **Registrador SaídaALU** contém a saída da ALU

⇒ O IR precisa conter a instrução até o fim da execução dessa instrução e, por isso, precisamos de um sinal de controle de escrita para esse registrador

⇒ Como uma única memória de dados é usada para instruções e para dados, precisamos de um mux para selecionar entre dois endereços: o PC (para instruções) e SaídaALU (para dados)

2. Escolhe entre o registrador B (Dado 2 lido do Banco de Registradores), a constante 4 (usada para incrementar o PC), o campo offset com sinal estendido (para endereços de memória em lw e sw) e o campo offset com sinal estendido e deslocado (para o cálculo do endereço de desvio)

Diagrama de um processador de 16 bits com arquitetura Harvard. O diagrama mostra o fluxo de dados entre o PC, a Memória, os Registradores de Instrução e de Dados, a ALU, e a Saída. O Registrador de Instrução recebe instruções da Memória e as distribui para os Registradores de leitura e escrita. O Registrador de Dados recebe dados da Memória e os distribui para a ALU e a Saída. A ALU realiza operações aritméticas e lógicas, com o resultado sendo enviado para a Saída e o Zero sendo enviado para o PC. O Registrador de Instrução também controla o Registrador de Dados e a ALU.

⇒ Como os multiplexadores e registradores adicionados são muito menores do que uma ALU e uma memória, já obtivemos uma redução no custo do hardware com esse esquema

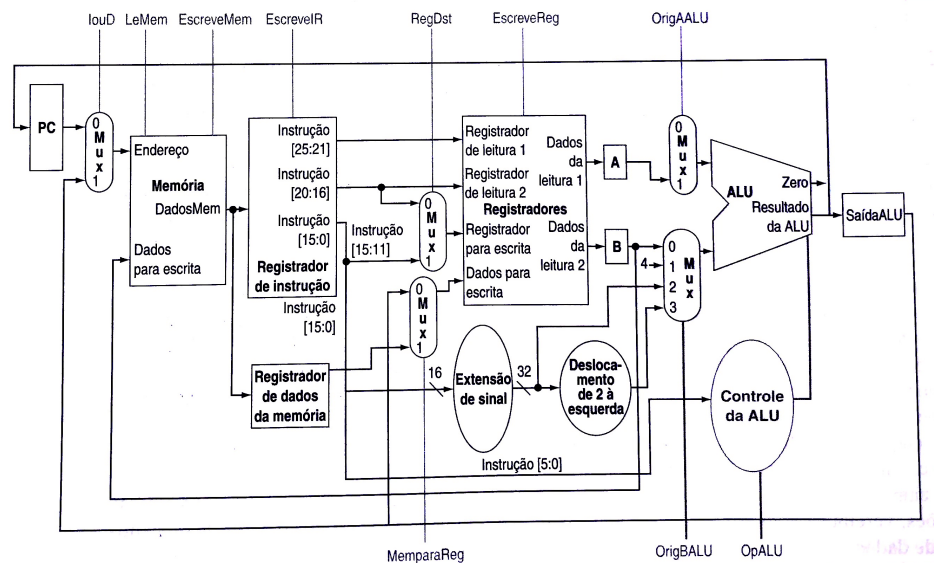
⇒ Como esse caminho de usa múltiplos ciclos de clock por instrução, ele exigirá um conjunto diferente de sinais de controle

⇒ PC, Memória, Banco de Registradores e o Registrador IR precisarão de sinais de controle de escrita; a Memória também precisa de um sinal de leitura

⇒ Podemos usar os sinais de controle da ALU do uniciclo para controlar a ALU do multiciclo

⇒ O mux de 4 entradas exige 2 linhas de controle (00, 01, 10, 11) para selecionar cada entrada

⇒ Abaixo temos a adição dos sinais de controle:



⇒ O caminho de dados do multiciclo apresentado ainda exige adições para suportar desvios e jumps

⇒ Com a instrução jump e branch, há 3 origens para o valor do PC:

1. A saída da ALU ($PC + 4$ durante a busca da instrução) que é armazenada diretamente no PC
2. O registrador SaídaALU, que é onde armazenamos o endereço de destino do desvio após ele ter sido calculado
3. Os 26 bits menos significativos do registrador IR deslocados 2 bits à esquerda e concatenados com os 4 bits mais significativos do PC incrementado, que é a instrução que deve ser executada em seguida quando usamos jump

⇒ O PC é escrito incondicionalmente quando se usa jump; O PC é escrito condicionalmente quando se usa beq → $PC + 4$ no PC é substituído por SaídaALU quando os 2 regs comparados são iguais → Precisamos de 2 sinais de controle separados em nossa implementação (1 para jump e 1 para beq):

⇒ **EscevePC**: causa uma escrita incondicional no PC

⇒ **EscevePCCond**: causa uma escrita no PC caso a condição seja verdadeira

⇒ Assim, colocamos a saída 0 da ALU e o sinal EscevePCCond como entradas de uma porta lógica AND e a saída dessa porta é a entrada de uma porta OR, da qual a outra entrada é EscevePC; a saída dessa porta OR é o

sinal de controle de escrita do PC

⇒ Caminho de dados do multiciclo com a unidade de controle:

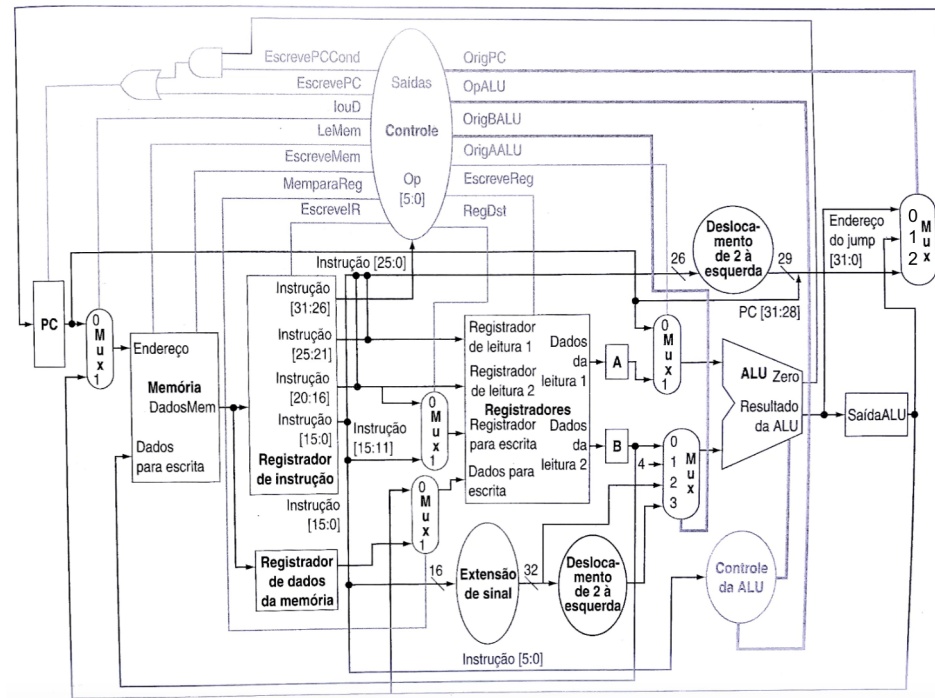


Figura 13: Caminho de dados completo para a implementação multiciclo com as linhas de controle necessárias

***Cuidado, nos slides de aula os nomes de alguns sinais estão diferentes da figura acima mas são equivalentes; peguei essa imagem do livro da 3ed para ficar coerente com os nomes na tabela abaixo e em outros lugares no resumo. Também foi feita uma modificação em relação a imagem do livro, na qual o mux que controla o dado que vai para o PC estava escrito 1 2 3 no mux e foi alterado para 0 1 2 a fim de ficar mais coerente**

⇒ Análise dos efeitos que os sinais de controle geram:

Ações dos sinais de controle de 1 bit		
Nome do sinal	Efeito quando inativo	Efeito quando ativo
RegDst	O número do registrador de destino do banco de registradores para a entrada Registrador para escrita vem do campo rt.	O número do registrador de destino do banco de registradores para a entrada Registrador para escrita vem do campo rd.
EscreveReg	Nenhum.	O registrador de uso geral selecionado pelo número na entrada Registrador para escrita é escrito com o valor da entrada Dados para escrita.
OrigAALU	O primeiro operando da ALU é o PC.	O primeiro operando da ALU vem do registrador A.
LeMem	Nenhum.	O conteúdo da memória no local especificado pela entrada Endereço é colocado na saída Dados da memória.
EscreveMem	Nenhum.	O conteúdo da memória no local especificado pela entrada Endereço é substituído pelo valor na entrada Dados para escrita.
MemparaReg	O valor enviado para a entrada Dados para escrita do banco de registradores vem de SaídaALU.	O valor enviado para a entrada Dados para escrita do banco de registradores vem do MDR.
loutD	O PC é usado para fornecer o endereço para a unidade de memória.	SaídaALU é usado para fornecer o endereço para a unidade de memória.
EscreveIR	Nenhum.	A saída da memória é escrita no IR.
EscrevePC	Nenhum.	O PC é escrito; a origem é controlada por OrigPC.
EscrevePCCond	Nenhum.	O PC é escrito se a saída Zero da ALU também estiver ativa.

***O nome do Sinal de Controle EscreveIR foi alterado na imagem da tabela acima pois estava escrito IR Write e para ficar coerente com a imagem do esquemático no resumo coloquei EscreveIR no lugar**

Ações dos sinais de controle de 2 bits		
Nome do sinal	Valor (binário)	Efeito
OpALU	00	A ALU realiza uma operação de adição.
	01	A ALU realiza uma operação de subtração.
	10	O campo funct da instrução determina a operação da ALU.
OrigBALU	00	A segunda entrada para a ALU vem do registrador B.
	01	A segunda entrada da ALU é a constante 4.
	10	A segunda entrada da ALU são os 16 bits menos significativos com sinal estendido do IR.
	11	A segunda entrada da ALU são os 16 bits menos significativos com sinal estendido do IR deslocados de 2 bits para a esquerda.
OrigPC	00	A saída da ALU (PC + 4) é enviada ao PC para escrita.
	01	O conteúdo de SaídaALU (o endereço de destino do desvio) é enviado ao PC para escrita.
	10	O endereço de destino do jump (IR[25:0]) deslocado de 2 bits para a esquerda e concatenado com PC + 4[31:28] é enviado ao PC para escrita.

5.1 Dividindo a execução da instrução em ciclos de clock

⇒ Dividimos o ciclo de clock para maximizar o desempenho

⇒ Podemos começar dividindo a execução de qualquer instrução em uma série de etapas, cada uma usando 1 ciclo de clock, buscando manter a quantidade de trabalho por ciclo aproximadamente igual

⇒ Por exemplo, iremos limitar cada etapa a conter no máximo 1 operação da ALU ou 1 acesso ao Banco de Registradores, ou 1 acesso a memória

⇒ Com essa restrição, o ciclo de clock pode ser tão curto quanto a mais longa dessas operações

⇒ A cada etapa, no final do ciclo de clock quaisquer valores de dados necessários em um ciclo subsequente precisam ser armazenados em um registrador, que pode ser um dos elementos de estado principais (o PC, o Banco de Registradores ou a Memória), um registrador temporário escrito em cada ciclo de clock (A, B, MDR ou SaídaALU) ou um registrador com controle de escrita (IR) → como o nosso projeto é acionado por transição de clock, podemos continuar a ler o valor de atual de um registrador; o novo valor não aparece até o próximo ciclo de clock

⇒ Muitos dos componentes do uniciclo operam em série, usando a saída de outro elemento como entrada; alguns operam em paralelo (ex: PC é incrementado e a instrução é lida ao mesmo tempo) → no multiciclo, todas as operações listadas em uma etapa ocorrem em paralelo dentro de 1 ciclo de clock, enquanto etapas sucessivas operam em série em diferentes ciclos de clock → a limitação de 1 operação na ALU, 1 acesso ao Banco de Registradores e 1 acesso à Memória determina o que pode entrar em uma etapa

⇒ Etapas de execução e suas ações (cada instrução do MIPS precisa de 3 a 5 dessas etapas):

1. Busca de instruções: buscar a instrução da memória e calcular o endereço da próxima instrução

IR $\leftarrow$ Memória[PC];
PC $\leftarrow$ PC + 4;

⇒ Operação: enviar o PC para Memória como endereço, realizar uma leitura e escrever a instrução no Registrador de instrução (IR)

⇒ Para implementar essa etapa: ativamos os sinais de controle LeMem e EscreveIR, IouD = 0 para selecionar o PC como a origem do endereço; como temos de incrementar o PC em 4 colocamos OrigAALU em 0 (enviamos o PC para a ALU), OrigBALU em 01 (4 para a ALU) e OPALU em 00 (soma); desejamos armazenar PC + 4 de volta no PC, assim colocamos OrigPC = 00 e ativamos EscrevePC

⇒ O incremento do PC e o acesso à memória de inst. podem ocorrer em paralelo → o novo valor do PC não é visível até o próximo ciclo de clock

2. Decodificação da Instrução e Busca dos Registradores:

⇒ Lemos os registradores indicados pelos campos rs e rt da instrução e armazenamos os valores lidos em A e B; também calculamos o endereço do desvio com a ALU, sendo o potencial destino do desvio salvo em SaídaALU

$$\begin{aligned}A &\leftarrow \text{Reg}[\text{IR}[25:21]]; \\B &\leftarrow \text{Reg}[\text{IR}[20:16]]; \\ \text{SaídaALU} &\leftarrow \text{PC} + (\text{estende-sinal}(\text{IR}[15:0]) \ll 2); \end{aligned}$$

⇒ Operação: acessar o Banco de Registradores para ler os registradores rs e rt e armazenar em A e B; calculamos o endereço de destino do desvio armazenando-o em SaídaALU, onde será usado se a instrução for de branch, por isso precisamos colocar OrigAALU = 0 (selecionar PC), OrigBALU = 11 (offset com sinal estendido e deslocado) e OpALU em 00 (soma)

⇒ Os acessos ao Banco de Registradores e o cálculo do destino do desvio ocorrem em paralelo

3. Execução, Cálculo do endereço de memória ou conclusão do desvio: primeiro ciclo durante o qual a operação do caminho de dados é determinada pela classe da instrução:

⇒ Referência à Memória:

$$\text{SaídaALU} \leftarrow A + (\text{estende-sinal}(\text{IR}[15:0]));$$

⇒ Operação: a ALU está somando os operandos para formar o endereço de memória; colocamos OrigAALU = 1 (seleciona dado no registrador A), OrigBALU = 10 (seleciona a saída da unidade de extensão de sinal) e OpALU = 00 (Soma)

⇒ Instrução Lógica ou Aritmética (Tipo R):

$$\text{SaídaALU} \leftarrow A \text{ op } B;$$

⇒ Operação: a ALU está realizando a operação especificada pelo opcode usando os dois valores em A e B lidos do Banco de Registradores no ciclo anterior; colocamos OrigAALU = 1 (seleciona dado no registrador A), OrigBALU = 00 (seleciona dado no registrador B) e OpALU = 10 (de modo que o campo funct determine as definições do sinal de controle da ALU)

⇒ Desvio(beq):

$$\text{if } (A == B) \text{ PC} \leftarrow \text{SaídaALU};$$

⇒ Operação: a ALU é usada para fazer a comparação de igualdade entre os dois regs lidos na etapa anterior e o sinal Zero da ALU é usado para determinar se o desvio deve ser tomado ou não; OrigAALU = 1 (registrador A), OrigBALU = 00 (registrador B) e OpALU = 01 (subtração); EscrevePCCond ativado para atualizar o PC (atualiza somente se a saída Zero da ALU for 1); OrigPC = 01 (seleciona SaídaALU que contém o endereço de desvio calculado na etapa anterior para ser o valor escrito no PC); Para desvios condicionais tomados, escrevemos no PC 2x: 1x durante a decodificação da instrução e 1x para o destino do desvio tomado, assim, o último valor escrito no PC é o valor usado para buscar a próxima instrução

⇒ Jump:

$$\text{PC} \leftarrow \{\text{PC}[31:28], (\text{IR}[25:0]), 2'b00\};$$

⇒ Operação: o PC é substituído pelo endereço do jump; OrigPC = 10 (seleciona endereço do Jump) e EscrevePC é ativado para escrever o endereço do jump no PC

4. Acesso à memória ou conclusão da instrução do tipo R:

⇒ Uma instrução lw/sw acessa a memória e uma instrução lógica ou aritmética escreve seu resultado

⇒ Quando um valor é lido da memória (lw) ele precisa ser armazenado no Registrador de dados (MDR), o qual precisa ser usado na próxima etapa

⇒ Referência à memória:

MDR $\leftarrow$ Memória [SaídaALU];

ou

Memória [SaídaALU] $\leftarrow$ B;

⇒ Operação: em qualquer caso (lw ou sw), o endereço de memória é calculado durante a etapa anterior e armazenado em SaídaALU; para sw, o operando que será armazenado na Memória é salvo em B (B é lido tanto na etapa 2 quanto na etapa 3; o mesmo valor é lido ambas as vezes já que o número do registrador não muda durante a execução - IR contém tal valor); LeMem (para lw) ou EscreveMem (para sw) precisará ser ativado; para lw/sw, IouD = 1 (seleciona endereço de memória da ALU ao invés do PC); como MDR é escrito a cada ciclo de clock, nenhum sinal para ele precisa ser ativado

⇒ Instrução Lógica ou Aritmética (tipo R):

Reg[IR[15:11]] $\leftarrow$ SaídaALU;

⇒ Operação: colocar o conteúdo de SaídaALU (valor calculado na etapa anterior) no registrador destino rd; RegDst = 1 (rd na instrução será usado para selecionar qual deve ser o registrador escrito no Banco de Registradores); EscreveReg precisa ser ativado e MemparaReg = 0 (seleciona dado em SaídaALU para ser escrito)

5. Conclusão da leitura da memória

⇒ Loads são completados escrevendo o valor lido da memória no Banco de Registradores

Reg[IR[20:16]] $\leftarrow$ MDR;

⇒ Operação: escrever os dados do lw, armazenados em MDR no ciclo anterior, no Banco de Registradores; RegDst = 0 (rt na instrução será usado para selecionar qual deve ser o registrador escrito no Banco de Registradores); EscreveReg precisa ser ativado e MemparaReg = 1 (seleciona dado em MDR para ser escrito)

⇒ Resumo:

Etapa	Ação para instruções tipo R	Ação para instruções de acesso à memória	Ação para desvios	Ação para jumps
Busca da instrução		IR $\leftarrow$ Memória[PC] PC $\leftarrow$ PC + 4		
Decodificação da instrução e busca dos registradores		A $\leftarrow$ Reg[IR[25:21]] B $\leftarrow$ Reg[IR[20:16]] SaídaALU $\leftarrow$ PC + (estende-sinal (IR[15:0]) << 2)		
Execução, cálculo do endereço ou conclusão do desvio/jump	SaídaALU $\leftarrow$ A op B	SaídaALU $\leftarrow$ A + estende-sinal (IR[15:0])	if (A == B) PC $\leftarrow$ SaídaALU	PC $\leftarrow$ {PC [31:28], (IR[25:0]), 2'b00}
Acesso à memória ou conclusão de instrução tipo R	Reg[IR[15:11]] $\leftarrow$ SaídaALU	Load: MDR $\leftarrow$ Memória[SaídaALU] ou Store: Memória [SaídaALU] $\leftarrow$ B		
Conclusão da leitura da memória		Load: Reg[IR[20:16]] $\leftarrow$ MDR		

5.2 Definindo o controle

⇒ Para o controle do caminho de dados multiciclo, o controle é mais complexo porque a instrução é executada em uma série de etapas → precisamos especificar os sinais a serem definidos em qualquer etapa e também a

próxima etapa da sequência

⇒ Vamos ver uma técnica baseada em **máquinas de estados finitos** para especificar o controle → conjunto de estados e diretrizes sobre como mudar de estado

⇒ As diretrizes são definidas por uma função de próximo estado, que mapeia o estado atual e as entradas para um novo estado

⇒ A implementação de uma máquina de estados finitos em geral considera que todas as saídas que não são explicitamente ativadas estão inativas → a correta operação do caminho de dados depende do fato de que um sinal que não é explicitamente ativado está inativo, em vez de agir como Don't Care

⇒ Na máquina de estados finitos quase sempre especificamos os sinais dos mux que nos interessam

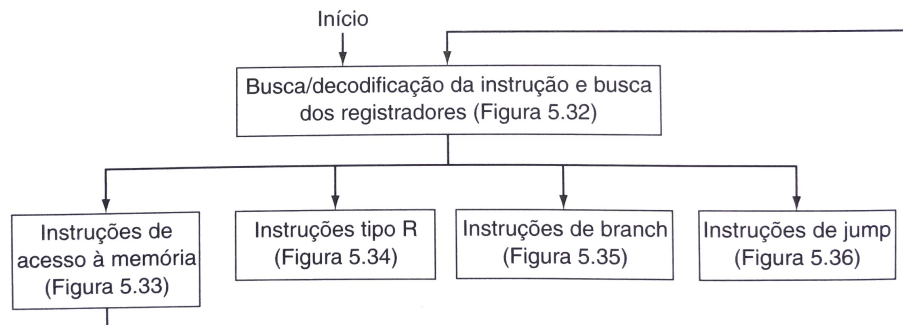
⇒ O controle de estados finitos basicamente corresponde às 5 etapas de execução mostradas anteriormente → cada estado da máquina de estados finitos usará 1 ciclo de clock → a máquina de estados finitos consistirá em várias partes

⇒ Como as 2 primeiras etapas da execução são idênticas para cada instrução, os dois estados iniciais da máquina de estados finitos serão comuns para todas as instruções

⇒ As etapas 3 a 5 diferem, dependendo do opcode

⇒ Após a execução da primeira etapa para uma determinada classe de instrução, a máquina de estados finitos retornará ao estado inicial para começar a buscar a próxima instrução

⇒ Visão de alto nível do controle da máquina de estados finitos:



⇒ Vamos mostrar os estados da máquinas de estados finitos usando uma representação tradicional; o estado 0 corresponde à etapa 1, o estado 1 corresponde à etapa 2,...

⇒ Os sinais ativados em cada estado são mostrados dentro do círculo que representa o estado; os arcos entre os estados definem o próximo estado e são rotulados com condições que selecionam um próximo estado específico quando vários próximos estados são possíveis

⇒ O processo de ramificação para diferentes estados dependendo da instrução é chamado de decodificação, uma vez que a escolha do próximo estado e, portanto, das ações que se seguem dependem da classe de instrução

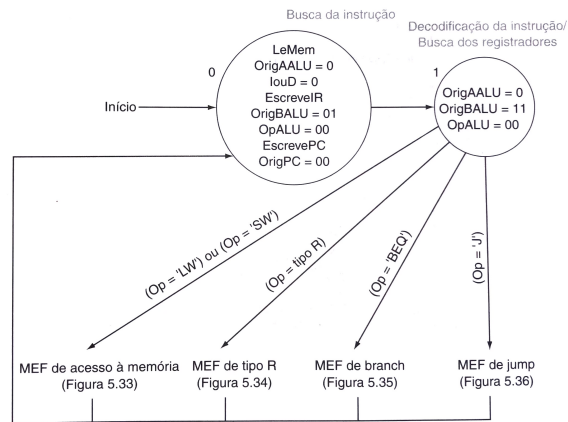


Figura 14: A parte da busca e decodificação da instrução é idêntica em todas as instruções

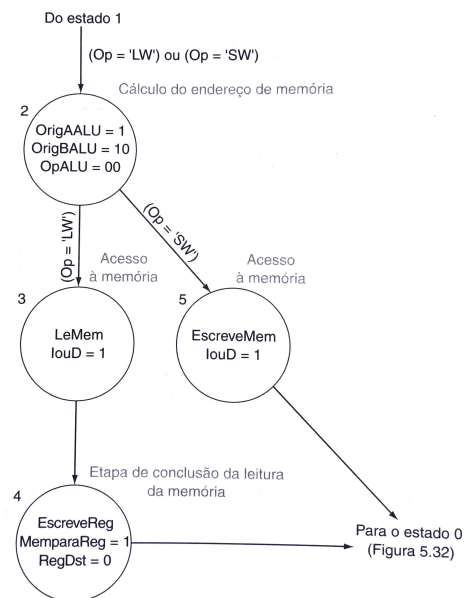


Figura 15: Máquina de estados finitos para controlar as instruções de referência à memória

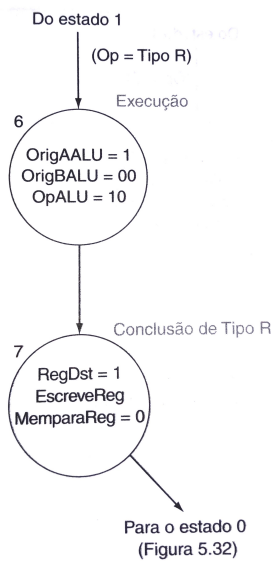


Figura 16: Máquina de estados finitos para controlar as instruções do tipo R

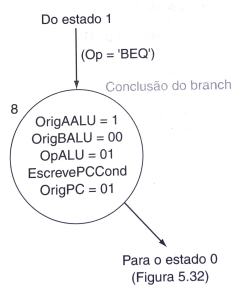


Figura 17: A instrução de branch requer um estado único

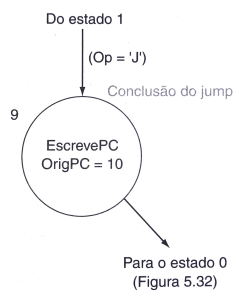


Figura 18: A instrução de jump requer um único estado que ative dois sinais de controle para escrever o PC com os 26 bits menos significativos do registrador de instrução deslocados 2 bits para a esquerda e concatenados com os 4 bits mais significativos do PC dessa instrução

⇒ Juntando todas as partes da máquina de estados finitos, formamos uma especificação para a unidade de controle, como se mostra abaixo:

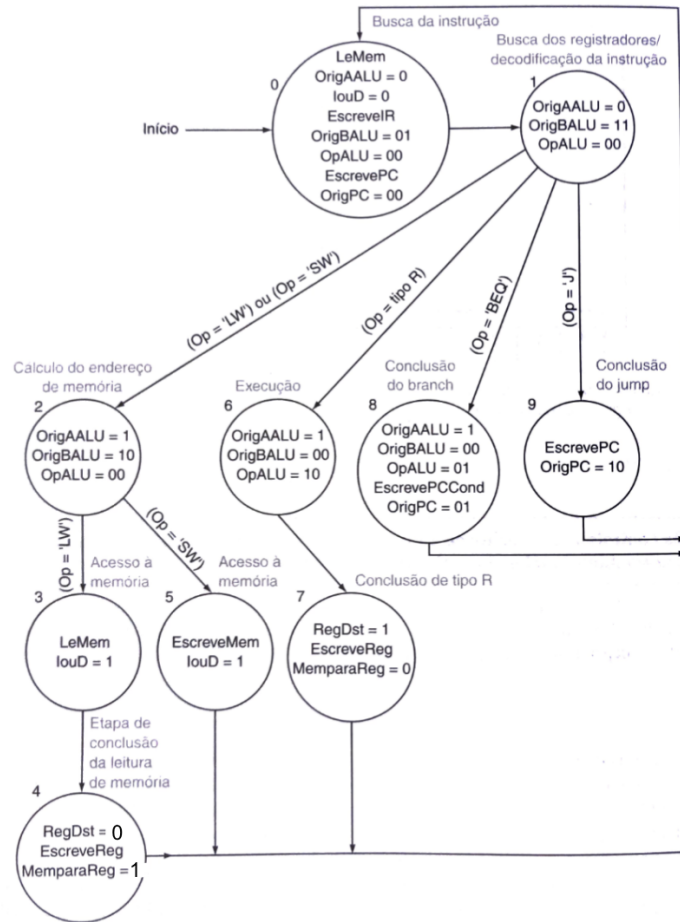
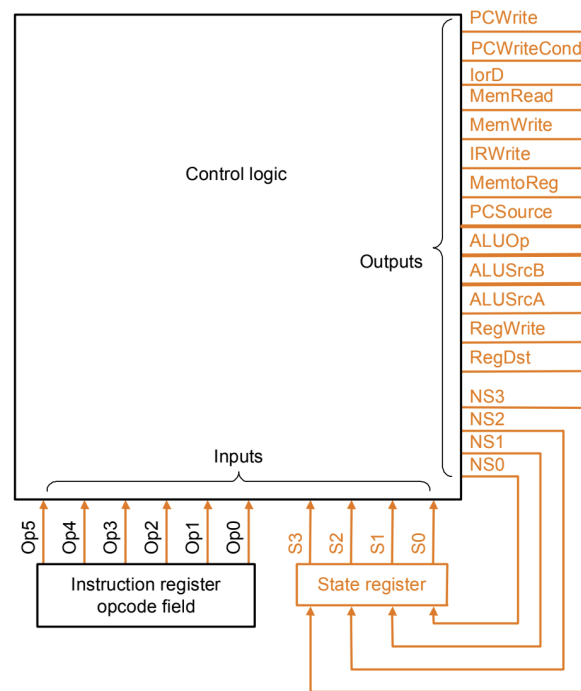
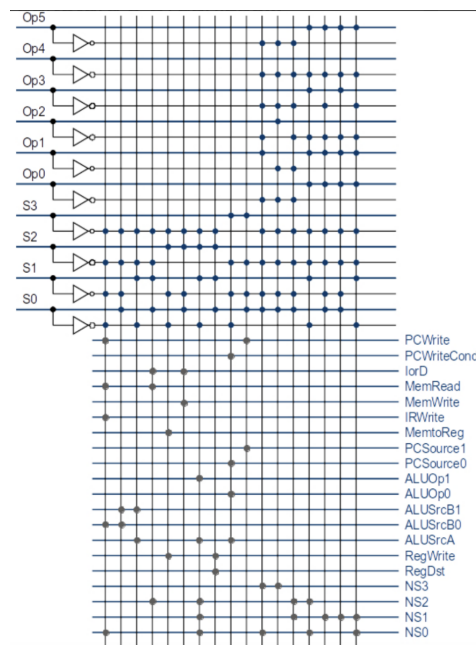


Figura 19: O controle completo para o caminho de dados mostrado na figura 13

⇒ Uma máquina de estados finitos pode ser implementada com um registrador temporário que contenha o estado atual e um bloco de lógica combinacional que determine os sinais do caminho de dados a serem ativados bem como o próximo estado:



⇒ Uma máquina de estados finitos pode ser implementada com um Implementação via PLA (Programmable Logic Array):



Referências

PATTERSON, David A.; HENNESSY, John L. ORGANIZAÇÃO E PROJETO DE COMPUTADORES: A INTERFACE HARDWARE/SOFTWARE. 4. ed. [S. l.]: CAMPUS, 2014.

PATTERSON, David A.; HENNESSY, John L. ORGANIZAÇÃO E PROJETO DE COMPUTADORES: A INTERFACE HARDWARE/SOFTWARE. 3. ed. [S. l.]: CAMPUS, 2014.

FERREIRA, Mateus Felipe Tymburibá. AOC1. 2022. Slides de aula.